UTD Student Life App

CE/CS/SE 3354 Software Engineering
Fall 2026

Team 9
Aakash Bhagavathi
Prakrit Chauhan
Aamir Khan
Hector Lopez
Parinita Chilukuri
Shayaan Rahim
William Turner

Project Goal

Help UTD students feel less lonely by helping them easily find others, events and study partners.

1

Connect with others easily

Search by name, major/interests and request to connect.

2

Create community

Post and RSVP to events like game nights and study sessions.

3

Secure

Only UTD emails allowed to sign up. Reports and Moderators.

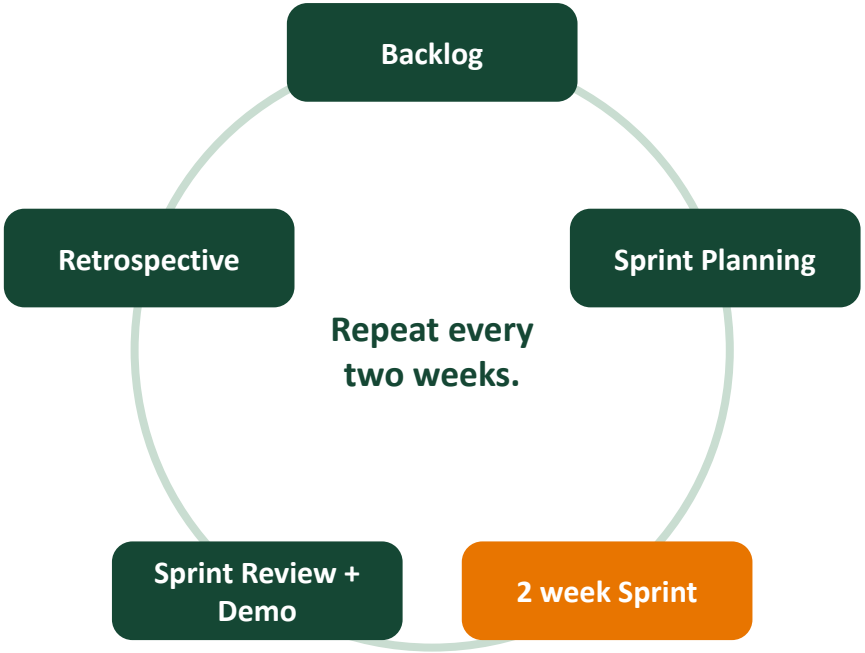
4

Support well-being

Easy access to UTD counselling.

Software Process

Agile / Scrum Methodology



Why we used Scrum

- We had an ever changing requirements list with extra requirements added from proposal feedback (FR-7 & FR-8 added).
- A short time frame with set milestones fit with sprint reviews.
- Features were shown during sprints to student testers.
- Our group was small and divided into specialties

Project Plan: Estimates

199

Function points (FP)

UFP: 179 x VAF: 1.11 = 199 FP.

25

Person-months (PM)

199 FP / 8 FP = 25 PM.

5mo

Timeframe

5 engineers for 100 working days = 5mo.

$322.4K

Cost

UTD price: $322.4K (+20%).

Cost	Amount	Description
Personnel	$260,337.60	4,000h @ $64.44h (BLS median) + 40h for training.
Hardware	$8,145.00	5 MacBooks Air, 4 Test Phones, Cloud Dev Server.
Software	$224.00	Apple Developer ($99), Google Play ($25), GitHub Team ($100).

Project Plan

Designed for a professional group of 5 between Jan 11th and June 1st 2027. Only working days and 8 hours per day.

Jan 11th - Jan 29th

Requirements & Design

Finalise SRS, UI Designs, Database Schema.

Feb 1st - Mar 26th

Sprint 1-4

Registration, OTP, Profiles, Events & RSVP.

Mar 29th - May 7th

Sprint 5-7

Connections, Messaging, Notifications, Moderation.

May 10th - June 1st

Testing & Deployment

System Testing, Security Testing, Deployment to App Store.

June - July: App Store & Google Play approval. Summer Beta with student testers. Live for Fall 2027 Welcome Week.

Functional Requirements

FR-1

Passwords are stored as Salted Hashes.

FR-2

Registration with UTD email and one time code.

FR-3

Search for students by name, major/interests.

FR-4

RSVP to events. Prevent if full.

FR-5

Report Abuse to Moderators.

FR-6

Create a profile with name, major/interests.

FR-7

Double opt-in requests.

FR-8

Direct message between connections.

FR-9

Create events with time and capacity.

Non Functional Requirements

Performance

Feed loads in under 2s. Reading posts under 800ms. 150 requests/sec.

Reliability

99% no crashes. 99.9% uptime.

Security

HTTPS/TLS for data in transit. Encrypted data at rest. All requests require permissions.

Usability

2 taps to make a post. Under 3 mins for first login.

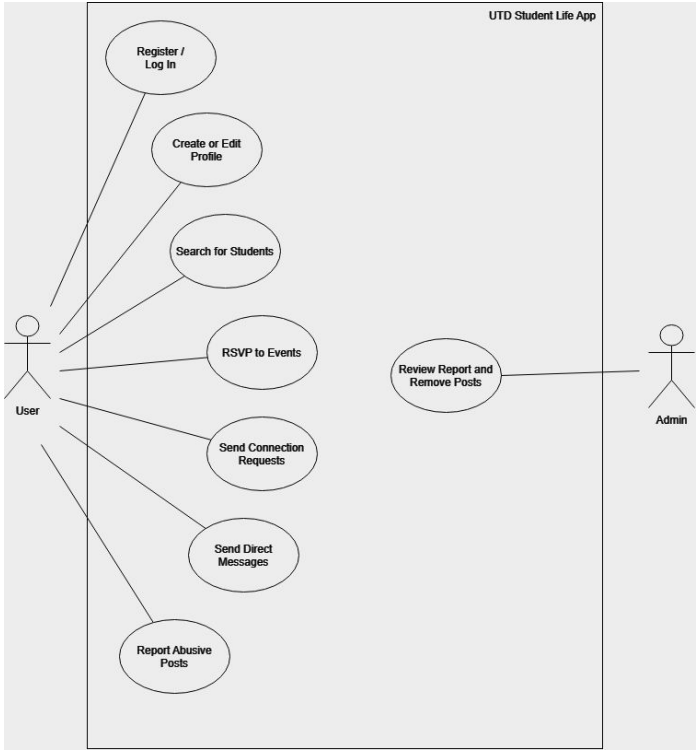
Environmental

iOS 16+ and Android 12+. Mobile only.

Legislative

FERPA, TDPSA, FTC Act compliant.

Use Case Modelling



Use Case Diagram.

Actors:

Student (User) will register, create profile, search for others, RSVP, connect, message and report.

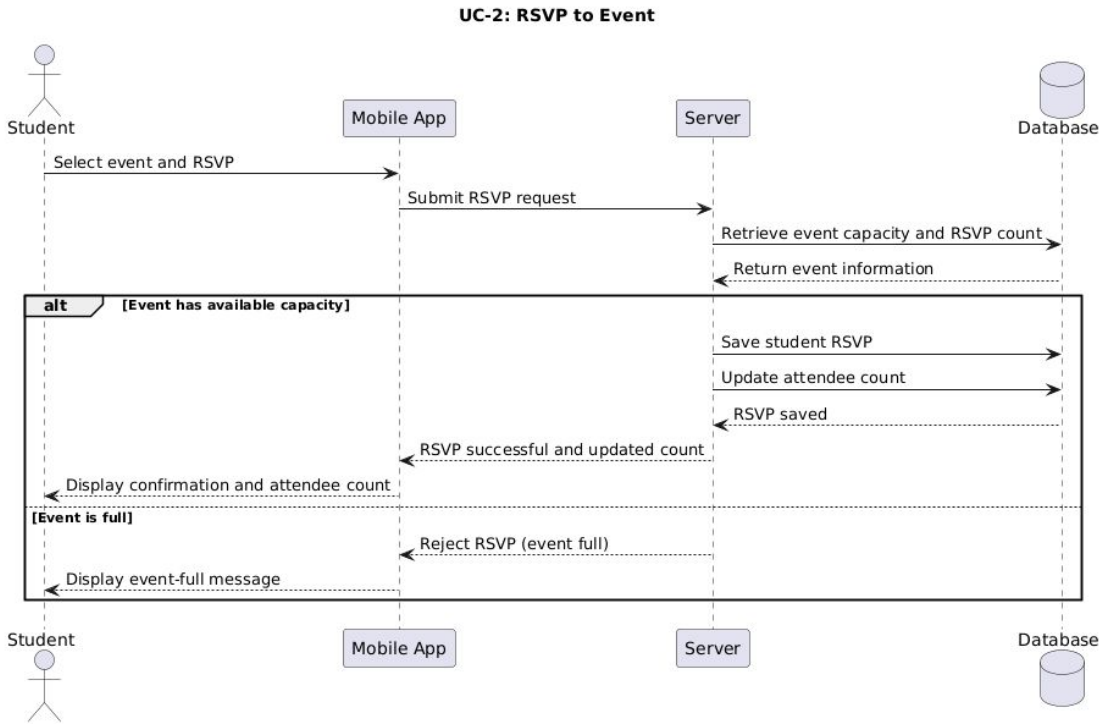
Moderator (Admin) will view reports, delete posts.

Use Case Example: UC-2 RSVP to Event

1. Open event and tap RSVP.
2. Check capacity.
3. If there is capacity save their RSVP and increase counter.
4. If there is no capacity do not allow RSVP.

Sequence Diagram

Sequence Diagram: UC-2 RSVP to Event

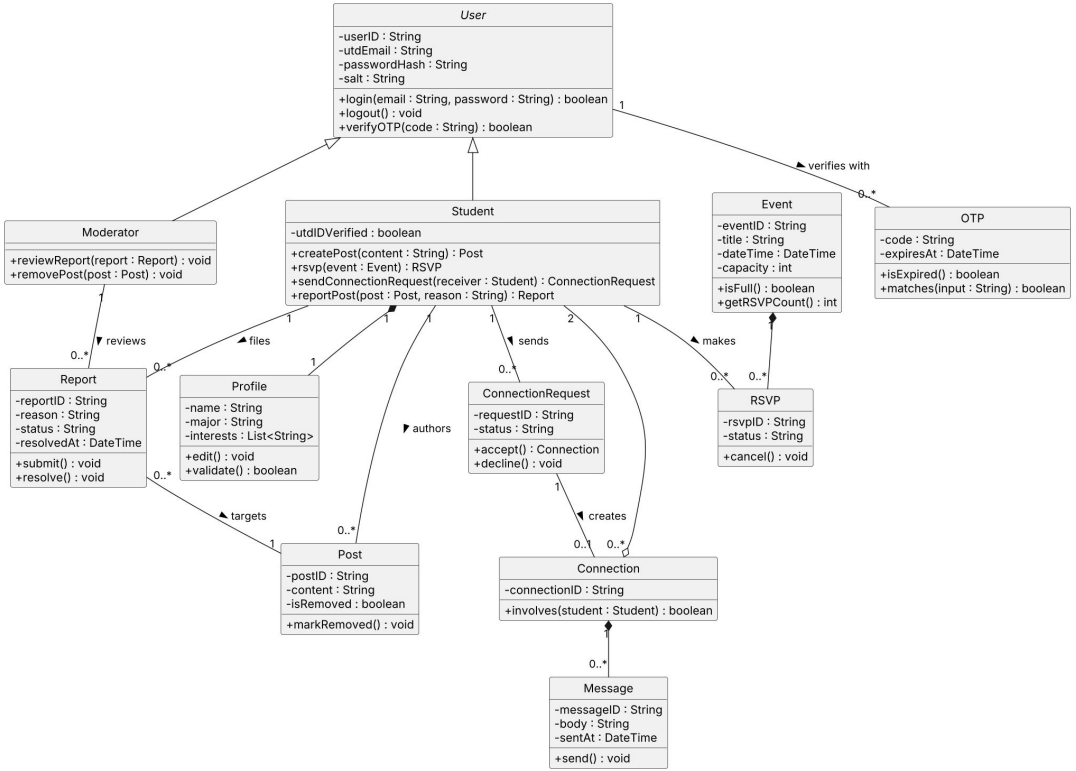


Student → Mobile App → Server → Database

Get event capacity and RSVP count.

```
if (Capacity > 0) {
  Save RSVP,
  Increase counter,
  Show confirmation
}
if (Event Full) {
  Do not save RSVP,
  Show Event Full
}
```

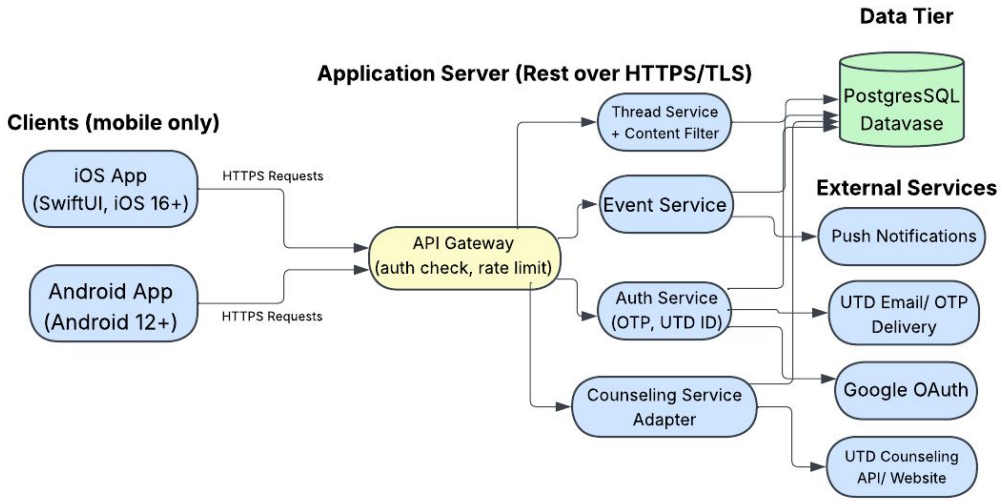
Class Diagram



Class Diagram.

- User is parent to Student/Moderator.
- Student creates Posts, RSVPs, Reports and ConnectionRequests.
- Event owns RSVPs and checks if full.
- Connection owns Messages so only connected students can message each other.
- OTP handles one time email code.

Architecture Design



Architecture Diagram.

SwiftUI (iOS) and Android clients. API Gateway. Thread, Event, Auth & Counseling Services. PostgreSQL.

Why we chose a client-server architecture:

- It is a mobile only app so all phones have the same client.
- All RSVPs and posts are in one database so they are consistent.
- Security policies are handled by the server.
- It is easier and cheaper than microservices.

Testing

Test Plan

1

What to Test

Event.addRSVP(), Event.isFull(), Event.getRSVPCount(), RSVP.cancel(). (FR-4 & UC-2)

2

Scope

Only testing RSVP logic on server side. Not testing UI, REST API or Database.

3

Method

Black box testing using Equivalence Partitioning and Boundary Values for capacity.

4

Tool

JUnit 4.13.2. Java 21. Automated so can be run after any changes.

Pass Criteria

- RSVP Saved if there is capacity.
- RSVP not saved if full.
- Cancel free up a space.
- No duplicates.
- No events with capacity < 1.

All 5 tests passed. 0 failures.

Test Cases & Analysis

ID	Input/Setup	Expected Outcome	Outcome
TC-1	Capacity = 2, 0 RSVPs. S001 RSVPs	Saved. Count = 1. Not full.	**Pass**
TC-2	Event Full (S001, S002). S003 RSVPs	EventFullException. Count = 2.	**Pass**
TC-3	Event Full. S001 Cancels. S003 RSVPs	Space freed. S003 Accepted. Count = 2.	**Pass**
TC-4	S001 RSVPs Again	Duplicate Exception.	**Pass**
TC-5	Create Event with capacity 0	IllegalArgumentException.	**Pass**

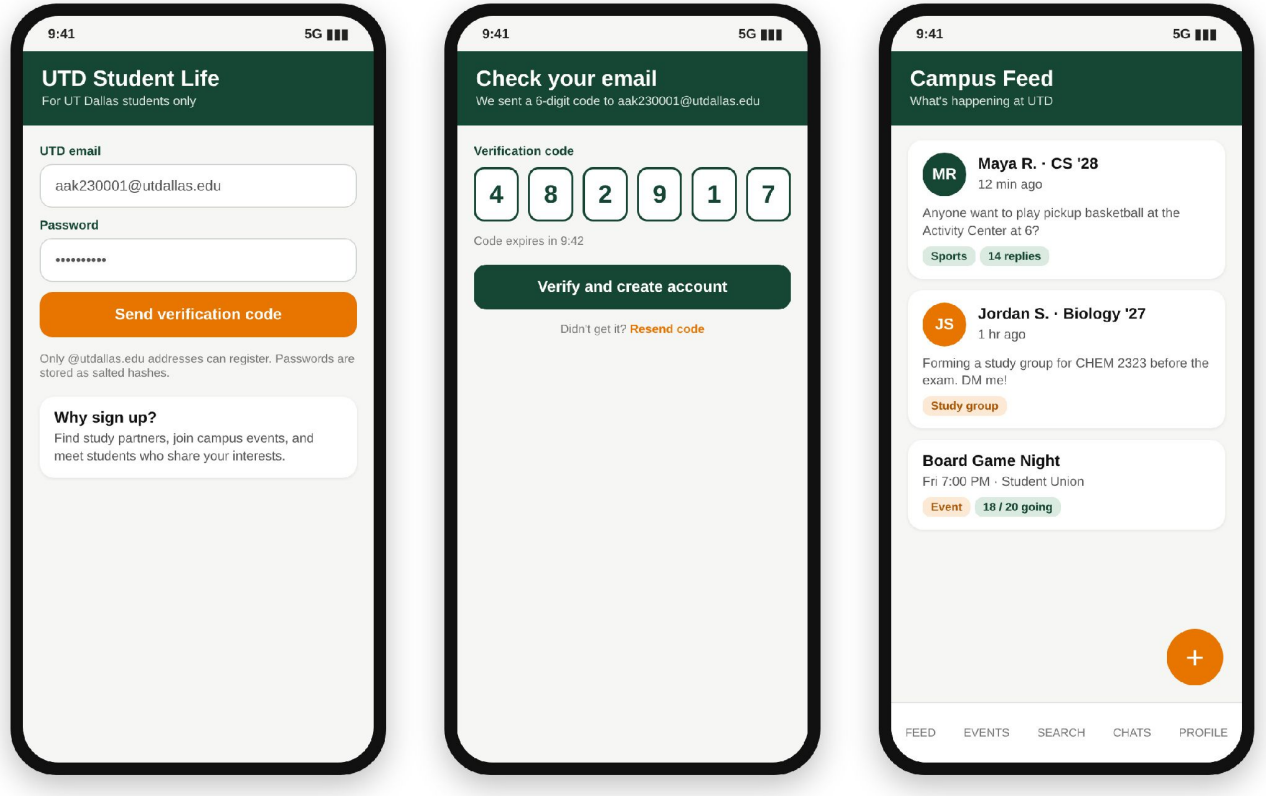
5/5

tests passed. Time taken = 26ms.

Analysis

Edge cases (full, cancelled with full, capacity 0) all worked as expected. Next we would test Auth & Messaging services in the same way.

Prototyping



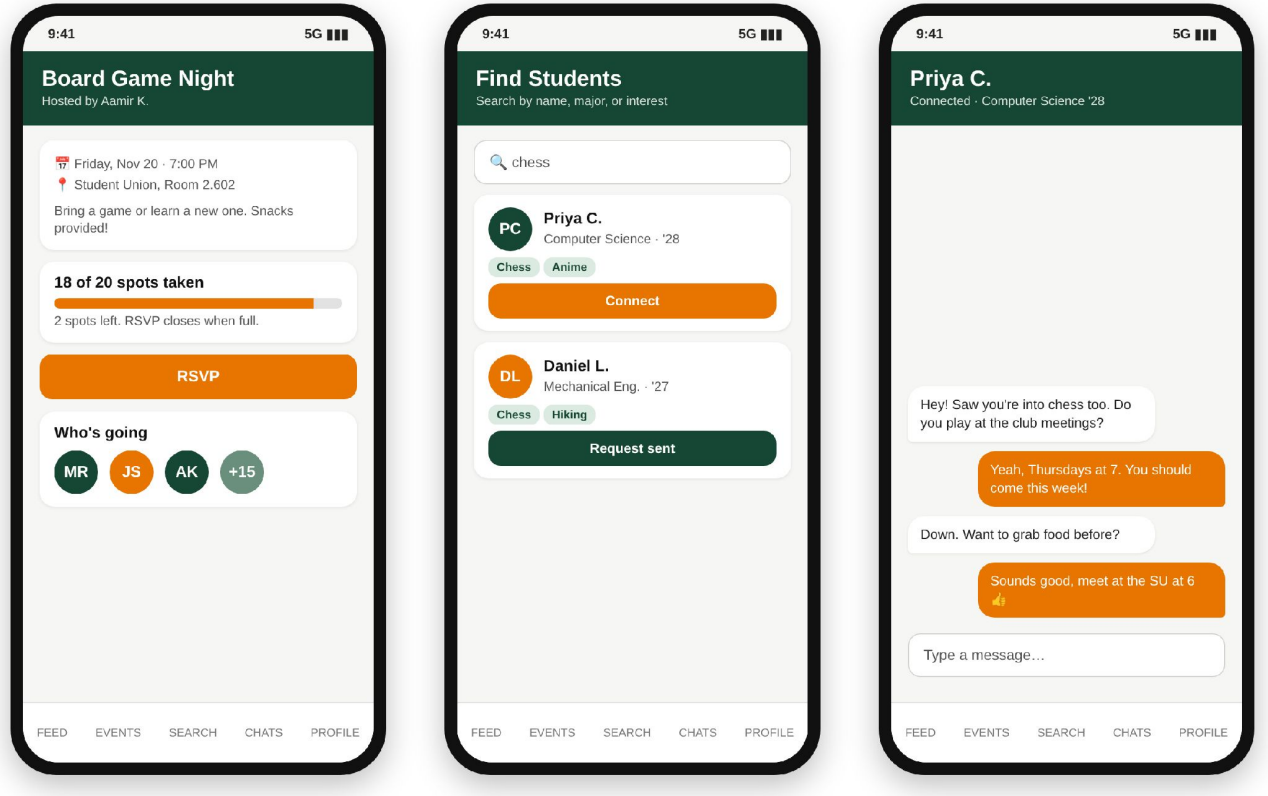
Prototype Screens.

Sign up
Only UTD emails (FR-2).

Verification
One time code via email.

Feed
Posts & Events. + button to create a post in 2 taps.

Prototyping



Prototype Screens.

Events
RSVP (Live counter. Closes if full. FR-4).

Search
Search by interest and request (FR-3 & FR-7).

Messaging
Only after accepted by both (FR-8).

Comparison with Similar Products

Feature	UTD app	Fizz	Bumble BFF	EngageUTD	Our App
Only verified students from UTD	✓	✓	—	✓	✓
Real Profiles (Name, Major, Interests)	—	—	✓	—	✓
Events created by Students. Capacity limit.	—	—	✓	—	✓
Double opt-in connections & DMs	—	—	✓	—	✓
Report posts & Moderators	—	✓	✓	—	✓

Our app is the only one which has a fully verified UTD community, real profiles, and events created by students with safe connections.

Lessons Learned

What went well:

- Divided work into specialties. All sections were progressing.
- Traceability matrix ensured each requirement had a design element and test.
- Use case diagrams made design easy to explain.

Challenges:

- Had to revise scope due to feedback from proposal.
- End to end encryption not realistic in 3 months so used TLS & encryption at rest.
- Managing 7 people and choosing a fair FP productivity rate.

Conclusion

- Private app for UTD students. Easily connect using events, interests and secure messaging.
- Complete SRS, design and estimated at 199 FP ($268.7K).
- Tested RSVP unit (all 5 tests passed).
- Next steps create remaining services, further automated testing and summer beta with students.